



More Complex Predicate







More Complex Predicate



**More Complex
Predicate**

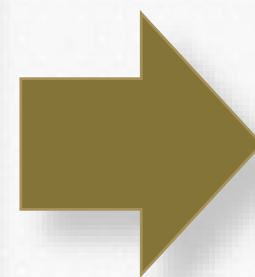
**IPredicate
Interface**

Need for More Complex Predicate

```
SELECT *  
FROM employees  
WHERE salary > 1000 AND age < 30;
```

BASE TABLE

ID	SALARY	AGE
1	500	20
2	1500	20
3	800	40
4	2000	40



FILTERED TABLE

ID	SALARY	AGE
2	1500	20

IPredicate Interface



Common interface for all predicate types

```
class IPredicate {  
public:  
    virtual ~IPredicate() = default;  
    virtual bool  
    check(const std::vector<std::unique_ptr<Field>> &tupleFields) const = 0;  
};
```

SimplePredicate: Comparison Operator



Implements the **IPredicate** interface for basic comparison operations

SALARY	>	1000
INDIRECT OPERAND	COMPARISON OPERATOR	DIRECT OPERAND

SimplePredicate: Comparison Operator



Implements the **IPredicate** interface for basic comparison operations

```
class SimplePredicate : public IPredicate {  
    Operand left_operand, right_operand;  
    ComparisonOperator comparison_operator;  
  
    bool  
    check(const std::vector<std::unique_ptr<Field>> &tupleFields) const override {  
        // Fetch fields based on operand type (DIRECT or INDIRECT)  
        // Compare fields based on the specified ComparisonOperator  
    }  
};
```

ComplexPredicate: Logical Operator



Implements the **IPredicate** interface for logical operations (AND/OR)

PREDICATE 1	AND/OR	PREDICATE 2
	LOGICAL OPERATOR	

ComplexPredicate: Logical Operator

```
class ComplexPredicate : public IPredicate {  
    std::vector<std::unique_ptr<IPredicate>> predicates;  
    LogicOperator logic_operator;  
  
    bool  
    check(const std::vector<std::unique_ptr<Field>> &tupleFields) const override {  
        // Evaluate all contained predicates based on the logic operator (AND, OR)  
    }  
};
```

Evaluating Complex Predicate

```
if (logic_operator == AND) {  
    for (const auto &pred : predicates) {  
        if (!pred->check(tupleFields)) {  
            return false; // All must pass  
        }  
    }  
    return true;  
}  
else if (logic_operator == OR) {  
    for (const auto &pred : predicates) {  
        if (pred->check(tupleFields)) {  
            return true; // Any one can pass  
        }  
    }  
    return false;  
}
```

Illustrative Complex Predicate

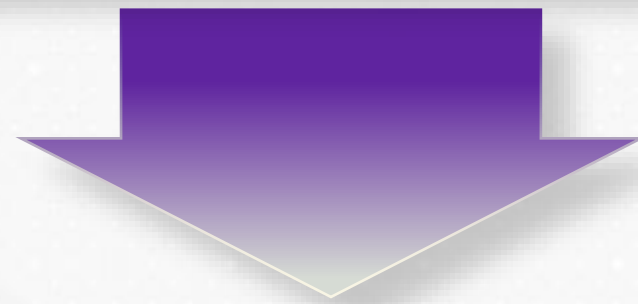
TUPLE [0]	>	2
INDIRECT OPERAND	COMPARISON OPERATOR	DIRECT OPERAND

TUPLE [0]	<	6
INDIRECT OPERAND	COMPARISON OPERATOR	DIRECT OPERAND

PREDICATE 1	AND	PREDICATE 2
IPREDICATE OBJECT POINTER	LOGICAL OPERATOR	IPREDICATE OBJECT POINTER

Illustrative Complex Predicate

TUPLE [0]	>	2
INDIRECT OPERAND	COMPARISON OPERATOR	DIRECT OPERAND



TUPLE [0]	<	6
INDIRECT OPERAND	COMPARISON OPERATOR	DIRECT OPERAND



PREDICATE 1	AND	PREDICATE 2
IPREDICATE OBJECT POINTER	LOGICAL OPERATOR	IPREDICATE OBJECT POINTER

Illustrative Complex Predicate

TUPLE [0]	>	2
INDIRECT OPERAND	COMPARISON OPERATOR	DIRECT OPERAND

TUPLE [0]	<	6
INDIRECT OPERAND	COMPARISON OPERATOR	DIRECT OPERAND

// Creating simple comparison predicates

```
auto greaterThanTwo = std::make_unique<SimplePredicate>(
    SimplePredicate::Operand(0),
    SimplePredicate::Operand(std::make_unique<Field>(2)),
    SimplePredicate::ComparisonOperator::GT);
```

```
auto lessThanSix = std::make_unique<SimplePredicate>(
    SimplePredicate::Operand(0),
    SimplePredicate::Operand(std::make_unique<Field>(6)),
    SimplePredicate::ComparisonOperator::LT);
```

Illustrative Complex Predicate

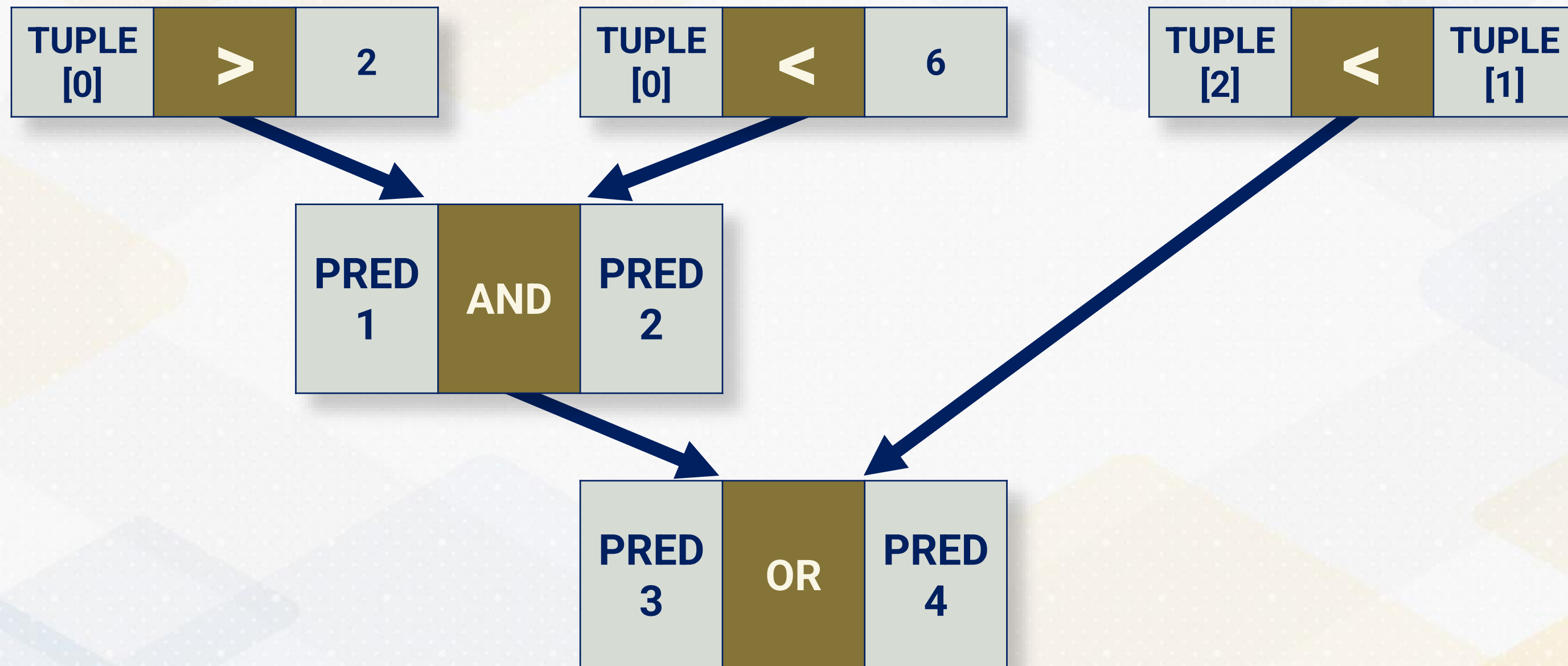
PREDICATE 1	AND	PREDICATE 2
IPREDICATE OBJECT POINTER	LOGICAL OPERATOR	IPREDICATE OBJECT POINTER

```
// Combining predicates into a complex predicate with AND logic
auto complexPredicate =
    std::make_unique<ComplexPredicate>(ComplexPredicate::LogicOperator::AND);

complexPredicate->addPredicate(std::move(greaterThanTwo));
complexPredicate->addPredicate(std::move(lessThanSix));

// Integrating ComplexPredicate with a SelectOperator for query processing
SelectOperator selectOperator(scanOperator, std::move(complexPredicate));
```

More Complex Predicate







More Complex Predicate



**More Complex
Predicate**

**IPredicate
Interface**

HASH AGGREGATION OPERATOR





Aggregation Query



**Aggregation
Query**

**Hash
Aggregation
Operator**

Aggregation Query

```
SELECT Category_ID,  
       SUM(Profit) AS Total_Profit  
FROM Sales_Data  
WHERE MONTH(DATE) = 'JUNE'  
GROUP BY Category_ID;
```

BASE TABLE

PRODUCT ID	PROFIT	CATEGORY ID	DATE
1	500	1	May 15
2	1000	1	June 10
3	1000	2	June 20
4	2000	2	June 30

HASH TABLE

KEY	VALUE
1	1000
2	3000

Hash Aggregation Operator

BASE TABLE

PRODUCT ID	PROFIT	CATEGORY ID	DATE
1	500	1	May 15
2	1000	1	June 10
3	1000	2	June 20
4	2000	2	June 30



HASH TABLE

KEY	VALUE
1	1000
2	3000

Hash Aggregation Operator

```
class HashAggregationOperator : public UnaryOperator {  
private:  
    std::vector<size_t> group_by_attrs;  
    std::vector<AggrFunc> aggr_funcs;  
    std::vector<Tuple> output_tuples;  
    ...  
};
```

Hash Aggregation Operator



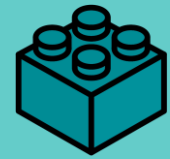
Unary Operator Class



Hash Aggregation

```
class HashAggregationOperator : public UnaryOperator {  
private:  
    std::vector<size_t> group_by_attrs;  
    std::vector<AggrFunc> aggr_funcs;  
    std::vector<Tuple> output_tuples;  
    ...  
};
```

Hash Aggregation Operator



Unary Operator Class



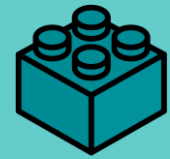
Hash Aggregation



group_by_attrs

```
class HashAggregationOperator : public UnaryOperator {  
private:  
    std::vector<size_t> group_by_attrs;  
    std::vector<AggrFunc> aggr_funcs;  
    std::vector<Tuple> output_tuples;  
    ...  
};
```

Hash Aggregation Operator



Unary Operator Class



Hash Aggregation



group_by_attrs



aggr_funcs vector

```
class HashAggregationOperator : public UnaryOperator {  
private:  
    std::vector<size_t> group_by_attrs;  
    std::vector<AggrFunc> aggr_funcs;  
    std::vector<Tuple> output_tuples;  
    ...  
};
```

Hash Aggregation Operator



Unary Operator Class



Hash Aggregation



group_by_attrs



aggr_funcs vector



output_tuples

```
class HashAggregationOperator : public UnaryOperator {  
private:  
    std::vector<size_t> group_by_attrs;  
    std::vector<AggrFunc> aggr_funcs;  
    std::vector<Tuple> output_tuples;  
    ...  
};
```

Hash Aggregation Operator

BASE TABLE

PRODUCT ID	PROFIT	CATEGORY ID	DATE
1	500	1	May 15
2	1000	1	June 10
3	1000	2	June 20
4	2000	2	June 30



HASH AGGREGATION OPERATOR

GROUP BY ATTRS	{3}
AGGREGATE FUNCTIONS	{SUM, 2}
OUTPUT TUPLES	{1, 1000}, {2, 3000}

Aggregation Logic

```
void HashAggregationOperator::open() {  
    std::unordered_map<std::vector<Field>, std::vector<Field>, FieldVectorHasher>  
        hash_table;  
    // Build the hash table with the aggregate tuples  
}  
  
bool HashAggregationOperator::next() {  
    // Iterate over the aggregate tuples in the hash table  
}
```

Aggregation Logic

```
void HashAggregationOperator::open() {  
    std::unordered_map<std::vector<Field>, std::vector<Field>, FieldVectorHasher>  
        hash_table;  
    // Build the hash table with the aggregate tuples  
}  
  
bool HashAggregationOperator::next() {  
    // Iterate over the aggregate tuples in the hash table  
}
```



Multi-Column Grouping

```
SELECT Category_ID,  
       MONTH(DATE),  
       SUM(Profit) AS Total_Profit  
FROM Sales_Data  
GROUP BY Category_ID, MONTH(DATE);
```

BASE TABLE

PRODUCT ID	PROFIT	CATEGORY ID	DATE
1	500	1	May 15
2	1000	1	June 10
3	1000	2	June 20
4	2000	2	June 30

AGGREGATED TABLE

CATEGORY ID	MONTH (DATE)	TOTAL PROFIT
1	May	500
1	June	1000
2	June	3000

Field Vector Hasher

```
struct FieldVectorHasher {  
    std::size_t operator()(const std::vector<Field> &fields) const {  
        std::size_t hash = 0;  
        for (const auto &field : fields) {  
            std::hash<std::string> hasher;  
            std::size_t fieldHash = 0;  
            fieldHash = hasher(...);  
            hash ^= fieldHash + 0x9e3779b9 + (hash << 6) + (hash >> 2);  
        }  
        return hash;  
    }  
};
```

Field Vector Hasher



Calculates
Hash Values

```
struct FieldVectorHasher {  
    std::size_t operator()(const std::vector<Field> &fields) const {  
        std::size_t hash = 0;  
        for (const auto &field : fields) {  
            std::hash<std::string> hasher;  
            std::size_t fieldHash = 0;  
            fieldHash = hasher(...);  
            hash ^= fieldHash + 0x9e3779b9 + (hash << 6) + (hash >> 2);  
        }  
        return hash;  
    }  
};
```

Hash Aggregation Operator

FIELD	OVERALL HASH VALUE	FIELD HASH VALUE

Hash Aggregation Operator

FIELD	OVERALL HASH VALUE	FIELD HASH VALUE
	0	

Hash Aggregation Operator

FIELD	OVERALL HASH VALUE	FIELD HASH VALUE
	0	
123	0x1f3...	std::hash("123")

Hash Aggregation Operator

FIELD	OVERALL HASH VALUE	FIELD HASH VALUE
	0	
123	0x1f3...	std::hash("123")
"May"	0x9e5...	std::hash("May")

Update Aggregate

```
SELECT Category_ID,  
       MONTH(DATE),  
       SUM(Profit) AS Total_Profit  
FROM Sales_Data  
GROUP BY Category_ID, MONTH(DATE);
```

BASE TABLE

PRODUCT ID	PROFIT	CATEGORY ID	DATE
1	500	1	May 15
2	1000	1	June 10
3	1000	2	June 20
4	2000	2	June 30

AGGREGATED TABLE

CATEGORY ID	MONTH (DATE)	TOTAL PROFIT
1	May	500
1	June	1000

Update Aggregate

```
SELECT Category_ID,  
       MONTH(DATE),  
       SUM(Profit) AS Total_Profit  
FROM Sales_Data  
GROUP BY Category_ID, MONTH(DATE);
```

BASE TABLE

AGGREGATED TABLE

PRODUCT ID	PROFIT	CATEGORY ID	DATE
1	500	1	May 15
2	1000	1	June 10
3	1000	2	June 20
4	2000	2	June 30



CATEGORY ID	MONTH (DATE)	TOTAL PROFIT
1	May	500
1	June	1000
2	June	1000

Update Aggregate

```
SELECT Category_ID,  
       MONTH(DATE),  
       SUM(Profit) AS Total_Profit  
FROM Sales_Data  
GROUP BY Category_ID, MONTH(DATE);
```

BASE TABLE

PRODUCT ID	PROFIT	CATEGORY ID	DATE
1	500	1	May 15
2	1000	1	June 10
3	1000	2	June 20
4	2000	2	June 30

AGGREGATED TABLE

CATEGORY ID	MONTH (DATE)	TOTAL PROFIT
1	May	500
1	June	1000
2	June	3000

Update Aggregate

```
Field updateAggregate(const AggrFunc &aggrFunc, const Field &currentAggr,  
                      const Field &newValue) {  
    // Check for type consistency  
    if (currentAggr.getType() != newValue.getType()) {  
        throw std::runtime_error("Mismatched Field types in aggregation.");  
    }  
    // Perform the specified aggregation operation  
    switch (aggrFunc.func) {  
    case AggrFuncType::SUM:  
        return Field(currentAggr.asInt() + newValue.asInt());  
    // other cases...  
    }  
}
```

Update Aggregate



```
Field updateAggregate(const AggrFunc &aggrFunc, const Field &currentAggr,
                      const Field &newValue) {
    // Check for type consistency
    if (currentAggr.getType() != newValue.getType()) {
        throw std::runtime_error("Mismatched Field types in aggregation.");
    }
    // Perform the specified aggregation operation
    switch (aggrFunc.func) {
    case AggrFuncType::SUM:
        return Field(currentAggr.asInt() + newValue.asInt());
    // other cases...
    }
}
```

Update Aggregate



```
Field updateAggregate(const AggrFunc &aggrFunc, const Field &currentAggr,
                     const Field &newValue) {
    // Check for type consistency
    if (currentAggr.getType() != newValue.getType()) {
        throw std::runtime_error("Mismatched Field types in aggregation.");
    }
    // Perform the specified aggregation operation
    switch (aggrFunc.func) {
    case AggrFuncType::SUM:
        return Field(currentAggr.asInt() + newValue.asInt());
    // other cases...
    }
}
```

getOutput

```
for (const auto &entry : hash_table) {  
    const auto &group_keys = entry.first;  
    const auto &aggr_values = entry.second;  
    for (const auto &key : group_keys) {  
        // Add group keys to the tuple  
        output_tuple.addField(std::make_unique<Field>(key));  
    }  
    for (const auto &value : aggr_values) {  
        // Add aggregated values to the tuple  
        output_tuple.addField(std::make_unique<Field>(value));  
    }  
}
```

getOutput

Appends Group Keys

```
for (const auto &entry : hash_table) {  
    const auto &group_keys = entry.first;  
    const auto &aggr_values = entry.second;  
    for (const auto &key : group_keys) {  
        // Add group keys to the tuple  
        output_tuple.addField(std::make_unique<Field>(key));  
    }  
    for (const auto &value : aggr_values) {  
        // Add aggregated values to the tuple  
        output_tuple.addField(std::make_unique<Field>(value));  
    }  
}
```

getOutput

Appends Group Keys

and Aggregate Value

```
for (const auto &entry : hash_table) {  
    const auto &group_keys = entry.first;  
    const auto &aggr_values = entry.second;  
    for (const auto &key : group_keys) {  
        // Add group keys to the tuple  
        output_tuple.addField(std::make_unique<Field>(key));  
    }  
    for (const auto &value : aggr_values) {  
        // Add aggregated values to the tuple  
        output_tuple.addField(std::make_unique<Field>(value));  
    }  
}
```

getOutput

Appends Group Keys

and Aggregate Value

➡ Aggregate Tuples

```
for (const auto &entry : hash_table) {  
    const auto &group_keys = entry.first;  
    const auto &aggr_values = entry.second;  
    for (const auto &key : group_keys) {  
        // Add group keys to the tuple  
        output_tuple.addField(std::make_unique<Field>(key));  
    }  
    for (const auto &value : aggr_values) {  
        // Add aggregated values to the tuple  
        output_tuple.addField(std::make_unique<Field>(value));  
    }  
}
```





Hash Aggregation Operator



**Hash
Aggregation
Operator**



**Aggregation
Query**



Complex Query







Relational Operator Framework



**Relational
Operator
Framework**



**Unix
Command
Pipeline**

Hard-Coded Query Function



```
SELECT column1, SUM(column2)
FROM tableName
WHERE column1 > 2 AND column1 < 7
GROUP BY column1;
```

Hard-Coded Query Function



Selects
Tuples
from
Table

```
SELECT column1, SUM(column2)
FROM tableName
WHERE column1 > 2 AND column1 < 7
GROUP BY column1;
```

Hard-Coded Query Function



Selects
Tuples
from
Table

Filters
Tuples
from
Column 1

```
SELECT column1, SUM(column2)
FROM tableName
WHERE column1 > 2 AND column1 < 7
GROUP BY column1;
```

Hard-Coded Query Function



Selects
Tuples
from
Table

Filters
Tuples
from
Column 1

Groups
Filters
Tuples

```
SELECT column1, SUM(column2)
FROM tableName
WHERE column1 > 2 AND column1 < 7
GROUP BY column1;
```

Hard-Coded Query Function



Selects
Tuples
from
Table

Filters
Tuples
from
Column 1

Groups
Filters
Tuples

Sums
Values of
Column 2

```
SELECT column1, SUM(column2)
FROM tableName
WHERE column1 > 2 AND column1 < 7
GROUP BY column1;
```

Hard-Coded Query Function

```
int main() {  
    db.scanTableToBuildIndex();  
  
    ...  
    int lowerBound = 2;  
    int upperBound = 7;  
    db.selectGroupBySum(lowerBound, upperBound);  
}
```

Hard-Coded Query Function

selectGroupBySum

```
int main() {  
    db.scanTableToBuildIndex();  
    ...  
    int lowerBound = 2;  
    int upperBound = 7;  
    db.selectGroupBySum(lowerBound, upperBound);  
}
```

Hard-Coded Query Function

selectGroupBySum

Scans Tuples &
Builds Index

```
int main() {  
    db.scanTableToBuildIndex();  
    ...  
    int lowerBound = 2;  
    int upperBound = 7;  
    db.selectGroupBySum(lowerBound, upperBound);  
}
```

Hard-Coded Query Function

selectGroupBySum

Scans Tuples &
Builds Index

Groups Filtered
Tuples

```
int main() {  
    db.scanTableToBuildIndex();  
    ...  
    int lowerBound = 2;  
    int upperBound = 7;  
    db.selectGroupBySum(lowerBound, upperBound);  
}
```

Hard-Coded Query Function

selectGroupBySum

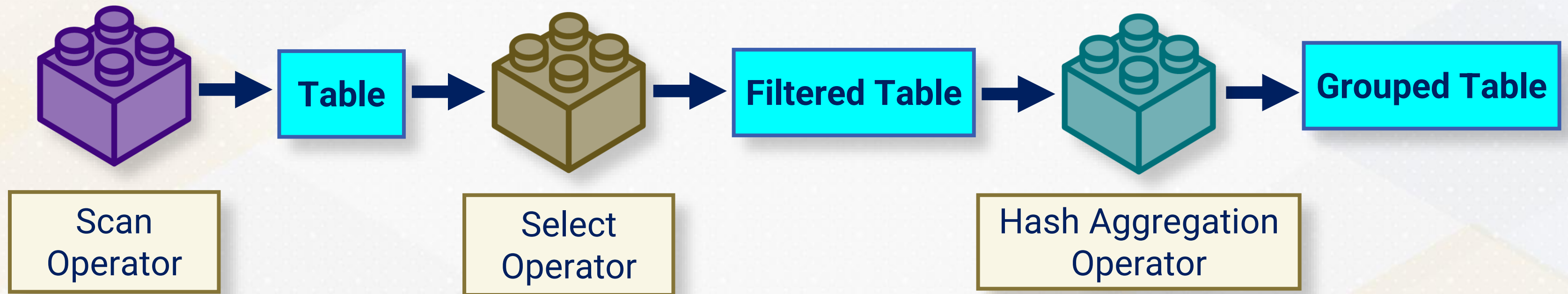
Scans Tuples &
Builds Index

Groups Filtered
Tuples

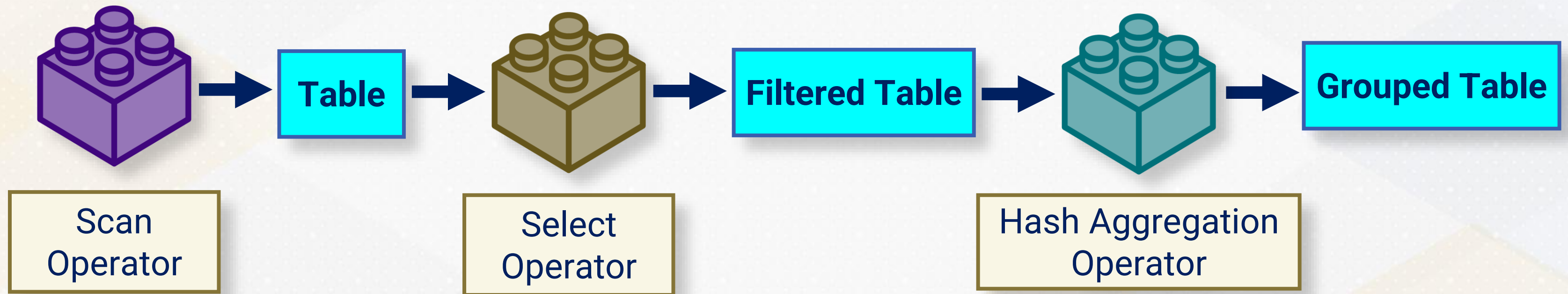
Sums Value of
Column 2

```
int main() {  
    db.scanTableToBuildIndex();  
    ...  
    int lowerBound = 2;  
    int upperBound = 7;  
    db.selectGroupBySum(lowerBound, upperBound);  
}
```

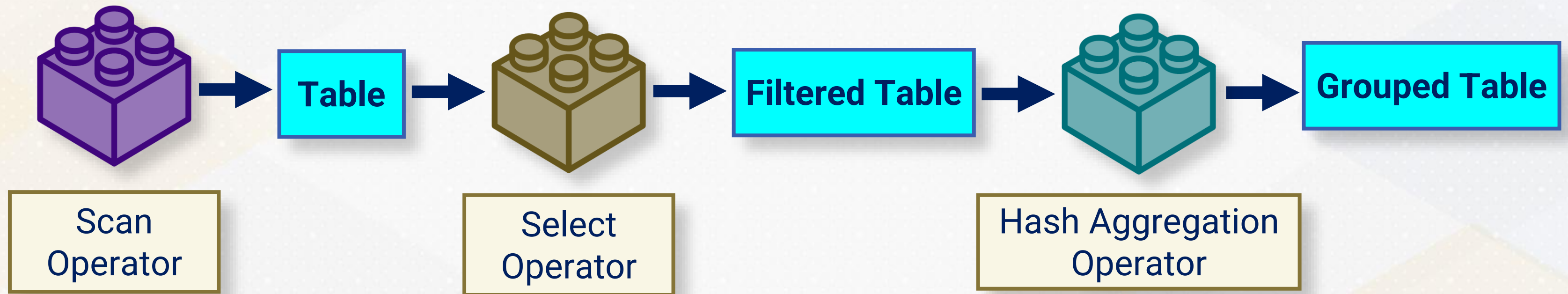
Relational Operator-Based Query Pipeline



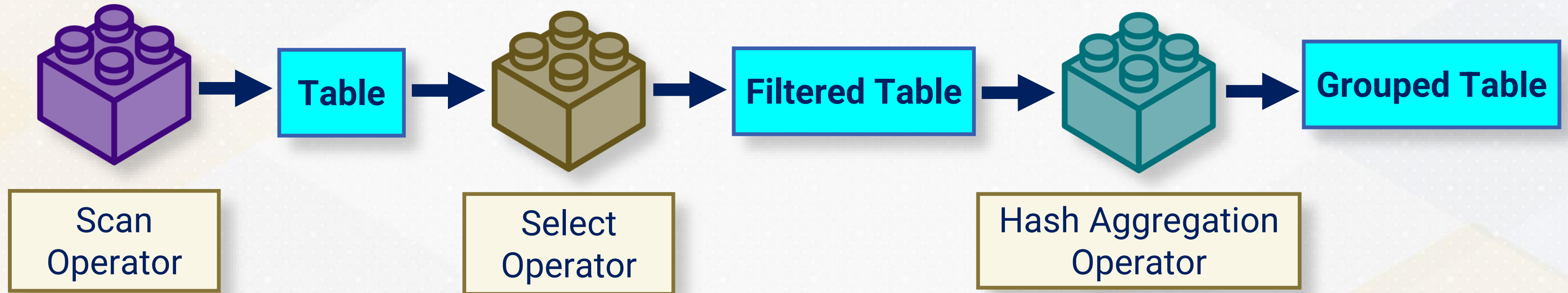
Relational Operator-Based Query Pipeline



Relational Operator-Based Query Pipeline



Relational Operator-Based Query Pipeline



Relational Operator-Based Query Pipeline

```
void executeQuery() {  
    ...  
    SelectOperator selectOperator(scanOperator, std::move(complexPredicate));  
    std::vector<AggrFunc> aggrFuncs{{AggrFuncType::SUM, 1}};  
    std::vector<size_t> groupByAttrs{0};  
    HashAggregationOperator hashAggregationOperator(selectOperator, groupByAttrs,  
                                                    aggrFuncs);  
    hashAggregationOperator.open();  
    while (hashAggregationOperator.next()) {  
        const auto &aggregatedFields = hashAggregationOperator.getOutput();  
        // Output key and aggregated value  
    }  
    hashAggregationOperator.close();  
}
```

Relational Operator-Based Query Pipeline



```
void executeQuery() {  
    ...  
    SelectOperator selectOperator(scanOperator, std::move(complexPredicate));  
    std::vector<AggrFunc> aggrFuncs{{AggrFuncType::SUM, 1}};  
    std::vector<size_t> groupByAttrs{0};  
    HashAggregationOperator hashAggregationOperator(selectOperator, groupByAttrs,  
                                                    aggrFuncs);  
    hashAggregationOperator.open();  
    while (hashAggregationOperator.next()) {  
        const auto &aggregatedFields = hashAggregationOperator.getOutput();  
        // Output key and aggregated value  
    }  
    hashAggregationOperator.close();  
}
```

Relational Operator-Based Query Pipeline

```
void executeQuery() {
```

```
...
```

```
SelectOperator selectOperator(scanOperator, std::move(complexPredicate));
```

```
std::vector<AggrFunc> aggrFuncs{{AggrFuncType::SUM, 1}};
```

```
std::vector<size_t> groupByAttrs{0};
```

```
HashAggregationOperator hashAggregationOperator(selectOperator, groupByAttrs,  
                                                  aggrFuncs);
```

```
hashAggregationOperator.open();
```

```
while (hashAggregationOperator.next()) {
```

```
    const auto &aggregatedFields = hashAggregationOperator.getOutput();
```

```
    // Output key and aggregated value
```

```
}
```

```
hashAggregationOperator.close();
```

```
}
```

Relational Operator-Based Query Pipeline

```
void executeQuery() {
```

```
...
```

```
SelectOperator selectOperator(scanOperator, std::move(complexPredicate));
```

```
std::vector<AggrFunc> aggrFuncs{{AggrFuncType::SUM, 1}};
```

```
std::vector<size_t> groupByAttrs{0};
```

```
HashAggregationOperator hashAggregationOperator(selectOperator, groupByAttrs,  
                                                  aggrFuncs);
```

```
hashAggregationOperator.open();
```

```
while (hashAggregationOperator.next()) {
```

```
    const auto &aggregatedFields = hashAggregationOperator.getOutput();
```

```
    // Output key and aggregated value
```

```
}
```

```
hashAggregationOperator.close();
```

```
}
```

Unix Operating System and Relational Database

	Unix Operating System	Relational Database
Timeline	1970s	1970s
Team	Dennis Ritchie, Ken Thompson, and others	Ted Codd, System R Team, and others
Location	AT&T Bell Labs	IBM

Unix Operating System and Relational Database

	Unix Operating System	Relational Database
Timeline	1970s	1970s
Team	Dennis Ritchie, Ken Thompson, and others	Ted Codd, System R Team, and others
Location	AT&T Bell Labs	IBM

Unix Operating System and Relational Database

	Unix Operating System	Relational Database
Timeline	1970s	1970s
Team	Dennis Ritchie, Ken Thompson, and others	Ted Codd, System R Team, and others
Location	AT&T Bell Labs	IBM

Unix Operating System and Relational Database

	Unix Operating System	Relational Database
Timeline	1970s	1970s
Team	Dennis Ritchie, Ken Thompson, and others	Ted Codd, System R Team, and others
Location	AT&T Bell Labs	IBM

Simplicity and Modularity

Each tool should do one and only one task well

// GREP TOOL

grep “error” file.txt

// SELECT OPERATOR

SELECT *

FROM TableName

WHERE Message LIKE '%error%';

Pipelining and Composability

// UNIX TOOL PIPELINE

```
cat file.txt | grep "error" | sort | uniq -c
```

// QUERY PIPELINE

```
SELECT ErrorMessage, COUNT(*) AS ErrorCount  
FROM LogTable  
WHERE ErrorMessage LIKE '%error%'  
GROUP BY ErrorMessage  
ORDER BY ErrorMessage;
```

Pipelining and Composability



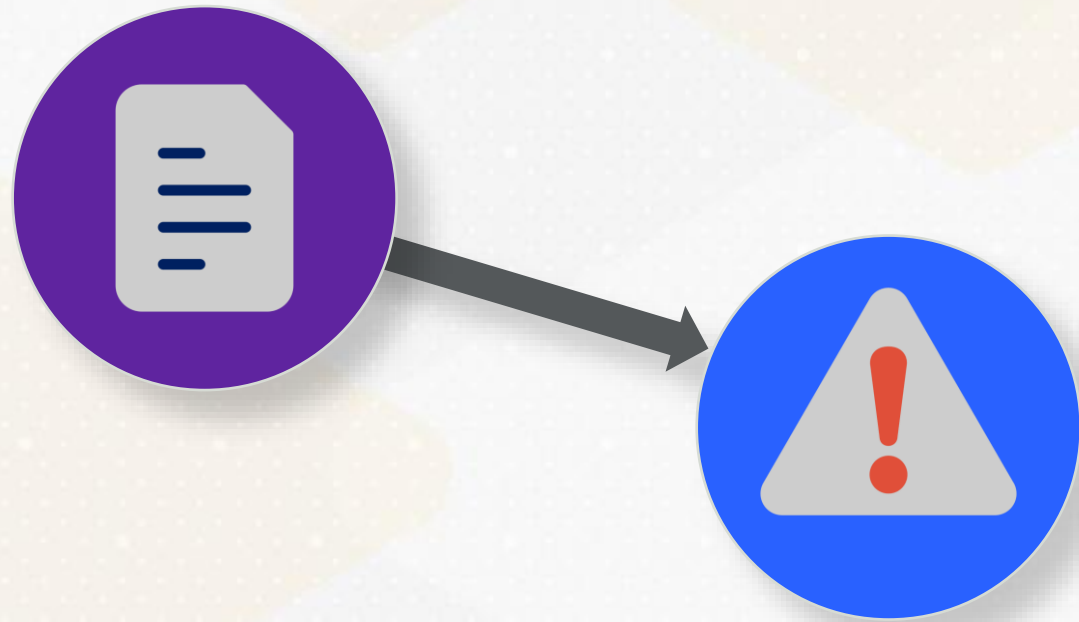
// UNIX TOOL PIPELINE

```
cat file.txt | grep "error" | sort | uniq -c
```

// QUERY PIPELINE

```
SELECT ErrorMessage, COUNT(*) AS ErrorCount  
FROM LogTable  
WHERE ErrorMessage LIKE '%error%'  
GROUP BY ErrorMessage  
ORDER BY ErrorMessage;
```

Pipelining and Composability



// UNIX TOOL PIPELINE

```
cat file.txt | grep "error" | sort | uniq -c
```

// QUERY PIPELINE

```
SELECT ErrorMessage, COUNT(*) AS ErrorCount  
FROM LogTable  
WHERE ErrorMessage LIKE '%error%'  
GROUP BY ErrorMessage  
ORDER BY ErrorMessage;
```

Pipelining and Composability



// UNIX TOOL PIPELINE

```
cat file.txt | grep "error" | sort | uniq -c
```

// QUERY PIPELINE

```
SELECT ErrorMessage, COUNT(*) AS ErrorCount  
FROM LogTable  
WHERE ErrorMessage LIKE '%error%'  
GROUP BY ErrorMessage  
ORDER BY ErrorMessage;
```

Pipelining and Composability



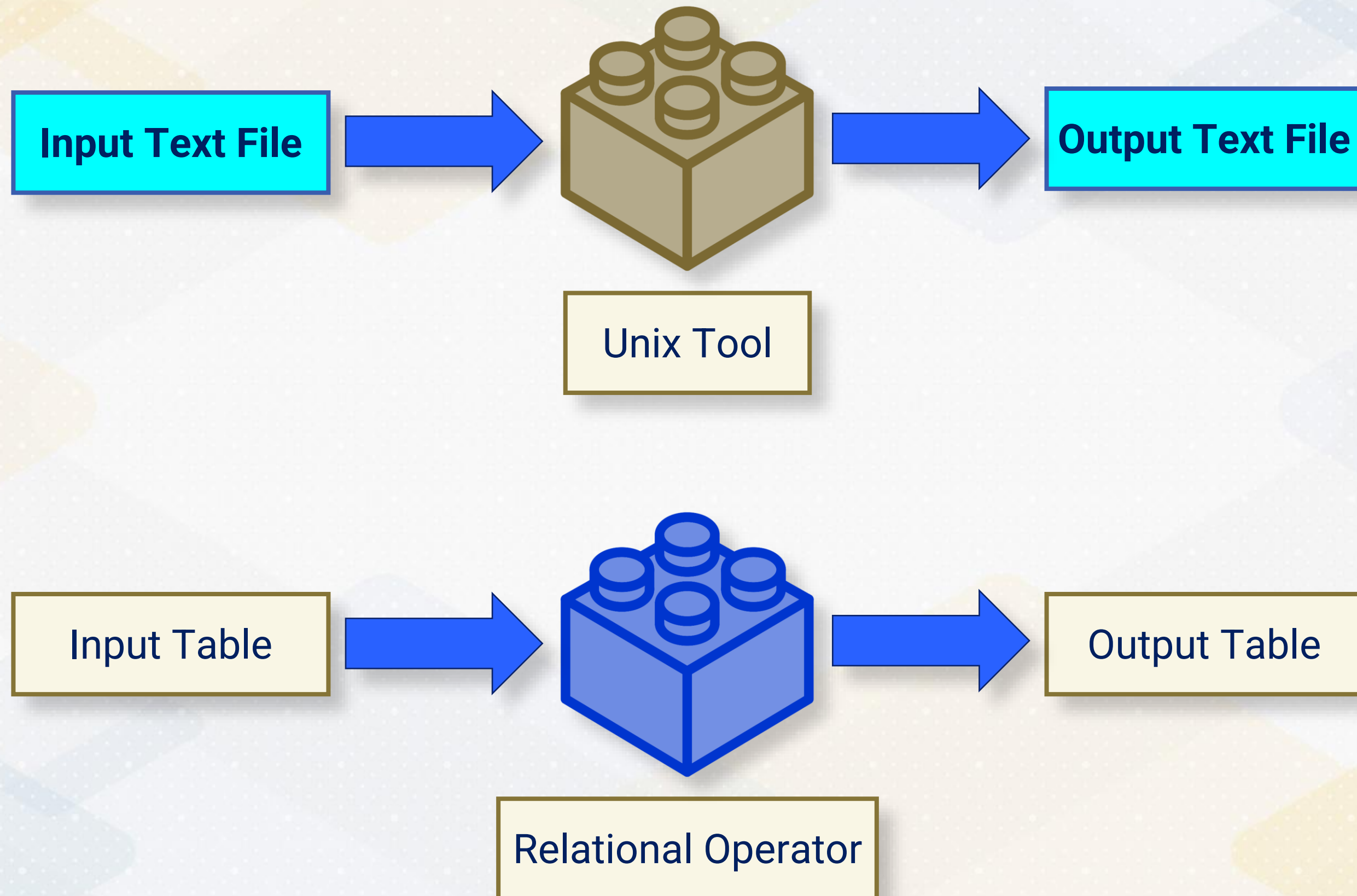
// UNIX TOOL PIPELINE

```
cat file.txt | grep "error" | sort | uniq -c
```

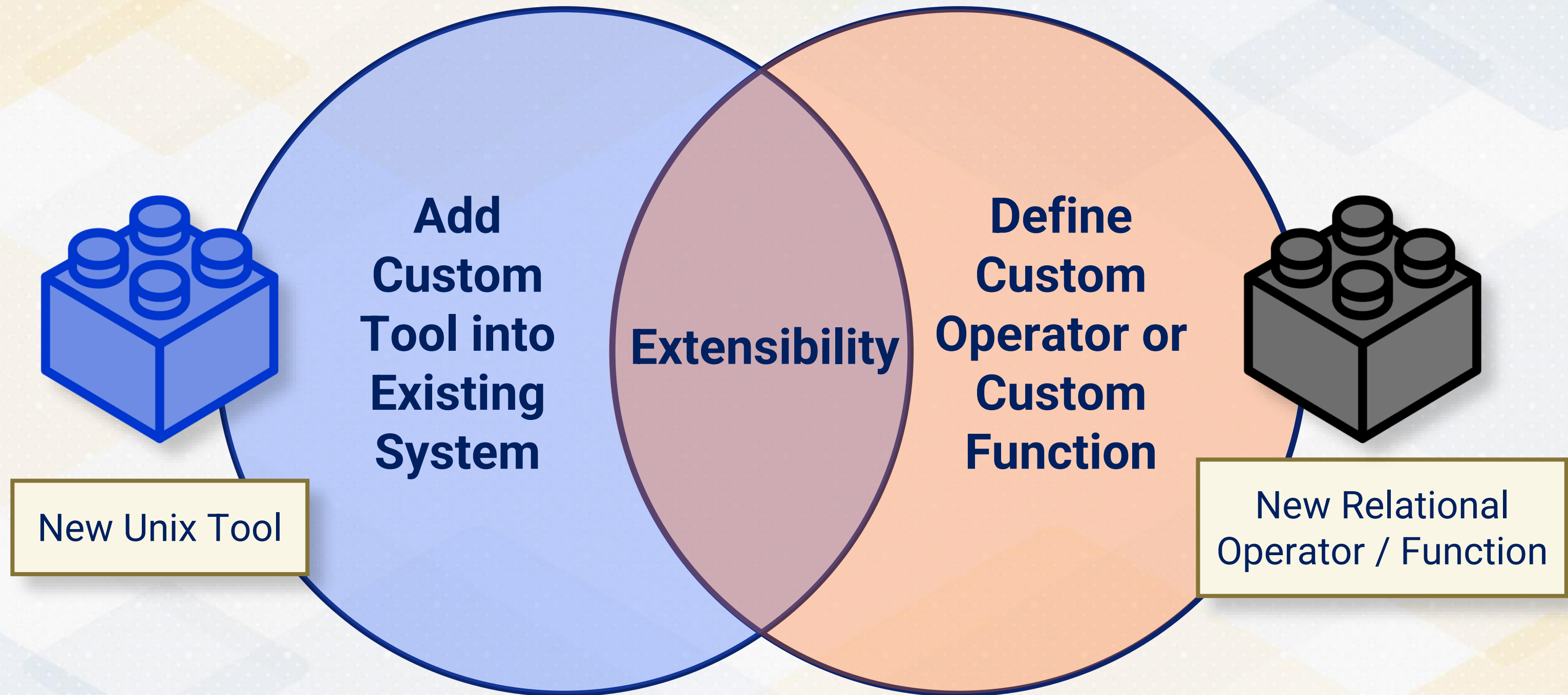
// QUERY PIPELINE

```
SELECT ErrorMessage, COUNT(*) AS ErrorCount  
FROM LogTable  
WHERE ErrorMessage LIKE '%error%'  
GROUP BY ErrorMessage  
ORDER BY ErrorMessage;
```

Standardized Interface



Extensibility







Relational Operator Framework



**Relational
Operator
Framework**



**Unix
Command
Pipeline**

Create Index Executor





Data Definition Language



**Data Definition
Language**



**Create Index
Executor**

SQL



Consists of Sub-Languages

Data Definition Language (DDL)	Data Manipulation Language (DML)

SQL



Consists of Sub-Languages

Data Definition Language (DDL)	Data Manipulation Language (DML)
Define or modify database design	
Create Table, Create Index, ...	
Drop Table, Drop Index, ...	

SQL



Consists of Sub-Languages

Data Definition Language (DDL)	Data Manipulation Language (DML)
Define or modify database design	Query or modify tuples in tables
Create Table, Create Index, ...	Select ...
Drop Table, Drop Index, ...	Update ...

Data Definition Language (DDL)

OPERATION	OBJECT

Data Definition Language (DDL)

OPERATION	OBJECT
CREATE, ALTER, DROP	

Data Definition Language (DDL)

OPERATION	OBJECT
CREATE, ALTER, DROP	TABLE, INDEX, VIEW, ...

Data Definition Language (DDL)

OPERATION	OBJECT
CREATE, ALTER, DROP	TABLE, INDEX, VIEW, ...

Tables

Primary
Data Structures
for Tuples

Data Definition Language (DDL)

OPERATION	OBJECT
CREATE, ALTER, DROP	TABLE, INDEX, VIEW, ...

Tables

Primary
Data Structures
for Tuples

Indexes

Derived Data
Structures

Data Definition Language (DDL)

OPERATION	OBJECT
CREATE, ALTER, DROP	TABLE, INDEX, VIEW, ...

Tables

Primary
Data Structures
for Tuples

Indexes

Derived Data
Structures

Views

Virtual
Tables

Create Index Operator



Builds index on a specific column

HASH INDEX
Column Offset: 1

EMP ID	SALARY	AGE
1	4000	35
2	5000	25
3	5000	55

KEY (SALARY)	VALUE (EMP ID)
4000	1
5000	2, 3

Create Index Executor

```
void open() override {  
    input->open();  
    while (input->next()) {  
        const auto &tuple = input->getOutput();  
        int key = tuple[attributeIndex]->asInt();  
        int value = tuple[1]->asInt(); // Example uses a fixed second attribute  
        hash_index.insertOrUpdate(key, value);  
    }  
}
```

Create Index Executor



```
void open() override {  
    input->open();  
    while (input->next()) {  
        const auto &tuple = input->getOutput();  
        int key = tuple[attributeIndex]->asInt();  
        int value = tuple[1]->asInt(); // Example uses a fixed second attribute  
        hash_index.insertOrUpdate(key, value);  
    }  
}
```

Create Index Executor



```
void open() override {  
    input->open();  
    while (input->next()) {  
        const auto &tuple = input->getOutput();  
        int key = tuple[attributeIndex]->asInt();  
        int value = tuple[1]->asInt(); // Example uses a fixed second attribute  
        hash_index.insertOrUpdate(key, value);  
    }  
}
```

Create Index Executor

```
bool next() override {  
    // Index creation is done in open(), so nothing to do here.  
    return false;  
}
```

Create Index Executor

```
bool next() override {  
    // Index creation is done in open(), so nothing to do here.  
    return false;  
}
```

Accessing the Index



Index accessed using getIndex()

```
const HashIndex &getIndex() const {  
    return hash_index;  
}
```

Create Index Operator Usage

```
// We want to index on the second attribute  
CreateIndexOperator createIndexOperator(scanOperator, 1);  
createIndexOperator.open(); // Builds the index  
createIndexOperator.close();  
  
const auto &index = createIndexOperator.getIndex();  
index.print(); // Displays the index contents
```

Create Index Operator Usage



```
// We want to index on the second attribute
CreateIndexOperator createIndexOperator(scanOperator, 1);
createIndexOperator.open(); // Builds the index
createIndexOperator.close();

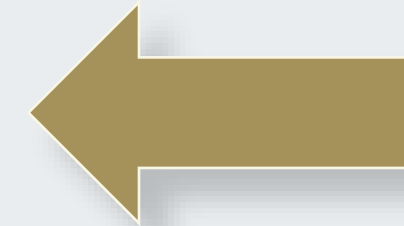
const auto &index = createIndexOperator.getIndex();
index.print(); // Displays the index contents
```

Create Index Operator Usage



```
// We want to index on the second attribute
CreateIndexOperator createIndexOperator(scanOperator, 1);
createIndexOperator.open(); // Builds the index
createIndexOperator.close();

const auto &index = createIndexOperator.getIndex();
index.print(); // Displays the index contents
```







Data Definition Language



**Data Definition
Language**



**Create Index
Executor**